Strata — Hướng dẫn Hub

Dành cho người vận hành hub

strata-kb v1.34.0 · 2026-11-01

1. Tài liệu này dành cho ai

Bạn vận hành hub. Đó là repository có thư mục `federation/` giữ bản duy nhất của mọi thứ mà tổ chức có thể tìm kiếm, và phục vụ bản đó cho con người, agent và CI. Bạn quyết định ai được publish, bạn merge những gì họ gửi lên, và bạn giữ cho dịch vụ chạy.

> **Không có gì mà repo child, BA hay Dev publish tìm kiếm được cho đến khi một pull request trên hub này được merge** (và hub đang chạy pull nó về). Bạn là cổng đó, trừ khi bạn bật auto-merge cho một repo (mục 4.3).

Hub có làm	Hub không làm
Giữ `federation/`, bản snapshot đã publish của mọi repo, và chỉ tìm kiếm trong đó	Tìm trong `.kb/` làm việc của chính nó: cho tới khi hub publish tài liệu của mình, server không tìm thấy chúng
Chạy MCP server HTTP, REST API và Web UI, và nhận publish từ CI (intake)	Cho repo chưa đăng ký publish khi `federation/registry.yaml` không còn rỗng
Review và merge PR publish; giữ tài khoản, registry, Q&A của ticket và claim ticket	Viết nội dung child, BA hay Dev thay họ: các repo đó tự biên soạn và tự publish

Repo hub có thể biên soạn tài liệu bằng đúng những bước như repo child. Các bước nằm trong tài liệu Child; tài liệu này không lặp lại (mục 3.5 nói về điểm khác trên hub). Repo BA và repo Dev publish lên cùng hub này và có tài liệu riêng: BA và Developer.

Slash command. Mọi slash command trong tài liệu này đều có cho Claude Code, GitHub Copilot và Cursor, trừ các ngoại lệ sau:

- `/kb-summarize` không có prompt cho Copilot. Copilot nạp `.github/instructions/kb-summarize.instructions.md` khi bạn làm việc trong `.kb/**`. Cursor còn có thêm một rule, `.cursor/rules/kb-summarize.mdc`.
- `/kb-ingest` và `/kb-publish` trong Claude Code là skill (`.claude/skills/<name>/`), không phải file trong `.claude/commands/`. Bạn vẫn gõ slash command như thường.
- `/kb-l3-fix` chỉ có ích khi tính năng sửa L3 bằng AI đang bật (tài liệu Child).
- Hub không có `/kb-mcp-setup` và không có `/kb-auto`.

Các lệnh hub có: `/kb-init`, `/kb-docker-setup`, `/kb-ingest`, `/kb-summarize`, `/kb-l3-fix`, `/kb-approve`, `/kb-publish`, `/kb-code-sync`.

---

2. Thiết lập lần đầu

2.1 Cài đặt và dựng khung

```
pip install strata-kb
mkdir my-hub && cd my-hub && git init
kb init --kind hub
```

Bạn cần Python 3.11 trở lên, Git và, cho dịch vụ, Docker. Chỉ thêm extra `ingest` (`pip install "strata-kb[ingest]"`) nếu bạn cũng biên soạn tài liệu trên máy này. Commit kết quả rồi đẩy lên một Git host: hub là một Git repository bình thường, và các publisher mở PR vào nó.

`kb init --kind hub` ghi `.kb/config.yaml` (`kind: hub`, `hub: "."` nên repo là hub của chính nó, `repo_id` lấy từ tên thư mục), `.kb/index.yaml`, `docker-compose.yml` và `.env.example`, workflow CI `.github/workflows/kb-publish.yml`, `.gitattributes` (kết thúc dòng LF) và `.gitignore` (giữ `.kb-work/`, `.kb-auth/`, `.kb-qa/` ngoài Git; `source/` và `sources/` có `.gitignore` riêng cho PDF nguồn), `QUICKSTART.md` cùng tám skill và command. `.mcp.json` và `.cursor/mcp.json` của nó chạy một MCP server **stdio** cho bạn (`python -m strata_kb.mcp --kb .kb/`), không phải server dùng chung. Chạy lại `kb init` là an toàn: nó tạo file còn thiếu và làm mới file dựng khung, nhưng bỏ qua `.kb/config.yaml` và `.kb/index.yaml` trừ khi bạn thêm `--force`. Thêm `--assets s3` để chọn lưu asset trên S3 (2.7).

2.2 Khởi động dịch vụ

```
kb docker-setup        # in your assistant: /kb-docker-setup
```

Lệnh ghi `.env` với một `STRATA_KB_HTTP_TOKEN` mới (48 ký tự hex, không bao giờ in ra), thêm `.env` vào `.gitignore` và chạy `docker compose up -d` khi Docker sẵn sàng. `--no-docker` chỉ chuẩn bị file và in các bước tiếp theo. Nếu `.env` đã có, lệnh dừng lại, trừ khi bạn thêm `--force` (trên terminal nó sẽ hỏi) để tạo lại token và giữ nguyên mọi dòng khác. Không dùng lệnh trợ giúp: `cp .env.example .env`, điền token của riêng bạn, `docker compose up -d`.

File compose chạy hai dịch vụ: `hub` (cổng 8321, `./` mount tại `/data`, một volume có tên cho cache model, health check `/api/health`) và `hub-sync` (lần quét code-sync hằng ngày, mục 4.4). Tag image là version của `kb` đã dựng khung. Để chạy không cần Docker:

```
export STRATA_KB_HTTP_TOKEN=$(python -c "import secrets; print(secrets.token_hex(24))")
python -m strata_kb.mcp --hub . --transport http --host 0.0.0.0
```

`--host` mặc định là `127.0.0.1` và `--port` là `8321`. Thêm extra `embed` (`pip install "strata-kb[embed]"`) để có nhánh semantic của tìm kiếm. Một unit systemd có trong `docs/deploy-remote-mcp.md`. Một process phục vụ bốn bề mặt:

Bề mặt	Dành cho	Xác thực
`/mcp`	AI agent	Bearer token
`/api/…`	script và tích hợp	Bearer token, hoặc cookie phiên
`/ui`	con người	tài khoản hub (2.4); bearer token mở UI ở chế độ chỉ đọc
`/intake/…`	publish từ CI	GitHub Actions OIDC; chỉ bật khi đã làm xong 2.5

Hub là bắt buộc: server lấy nó từ `--hub`, rồi `STRATA_KB_HUB`, rồi `.kb/config.yaml`, và từ chối khởi động nếu không có.

Token. Server HTTP từ chối khởi động nếu thiếu `STRATA_KB_HTTP_TOKEN`, và từ chối giá trị mẫu `change-me`, token ngắn hơn 32 ký tự, hoặc token không phải ASCII in được, không có dấu cách. Token phục vụ `/api`, `/mcp` và CI; con người đăng nhập `/ui` bằng tài khoản.

2.3 Trước khi mở ra ngoài

- **Thay token được sinh sẵn.** `kb docker-setup` sinh một token cho tiện. Hãy coi nó là giá trị tạm: đặt bí mật của riêng bạn vào `.env` và giữ trong một trình quản lý bí mật. Mọi repo kết nối qua MCP đều giữ token này. Sau mỗi thay đổi `.env`, tạo lại dịch vụ: `docker compose up -d --force-recreate` (mọi dịch vụ: `hub-sync` đọc cùng `.env`).
- **Sau reverse proxy, hãy đặt `STRATA_KB_TRUSTED_PROXIES`** bằng số proxy tin cậy đứng trước server (số nguyên, mặc định `0`; giá trị khác làm server không khởi động). Để `0` sau proxy thì mọi người gọi đều trông như proxy: một người gọi có thể làm cạn bộ giới hạn tốc độ của tất cả, và một người bị khoá đăng nhập khoá luôn mọi người. Khi đã đặt, nó còn cho cookie phiên `/ui` có cờ `Secure` dựa trên `X-Forwarded-Proto`; chưa đặt thì server ghi cảnh báo và đặt cookie không có `Secure`. Mỗi proxy phải nối thêm vào `X-Forwarded-For` và `X-Forwarded-Proto`, không được ghi đè. Từ `1` trở lên, hãy bind server sao cho chỉ proxy của bạn tới được: client kết nối thẳng sẽ tự điều khiển khoá giới hạn tốc độ của nó.
- **Đặt `STRATA_KB_ALLOWED_HOSTS`** bằng (các) tên host công khai mà proxy phục vụ, phân cách bằng dấu phẩy, chỉ tên trần (`kb.example.com`; có scheme, port hay path thì server không khởi động). Chưa đặt, `/mcp` sau proxy ở một tên thật trả `421 Invalid Host header` trong khi `/api` và `/ui` vẫn chạy.
- **Biết giới hạn.** Xác thực HTTP là bearer token cho `/api`, `/mcp` và CI, cộng tài khoản hub cục bộ cho UI: không có OAuth hay SSO. Vậy là đủ cho mạng nội bộ và chưa sẵn sàng cho internet công cộng. TLS thuộc về proxy, HSTS cũng vậy: sau proxy server không bao giờ thấy https nên nó không gửi HSTS.

Hướng dẫn triển khai đầy đủ nằm trong `docs/deploy-remote-mcp.md`.

2.4 Tạo admin đầu tiên

Con người đăng nhập `/ui` bằng tên người dùng và mật khẩu. Tài khoản nằm trong `<hub>/.kb-auth/` (`users.db` và `session.key`). **Không bao giờ commit thư mục đó**: `.gitignore` của hub mới đã liệt kê nó và hub đang chạy cũng thêm nó vào `.git/info/exclude`. Xoá `session.key` sẽ đăng xuất mọi người. Chọn một cách để tạo tài khoản đầu tiên:

- **Trang thiết lập (mặc định).** Mở hub trong trình duyệt. Khi chưa có tài khoản nào, mọi trang `/ui` chuyển hướng sang `/ui/setup`. Đọc mã thiết lập dùng một lần trong log của hub (`docker compose logs hub`; với systemd là `journalctl -u <unit>`), nhập nó cùng tên người dùng và mật khẩu, và tài khoản đó trở thành `admin`. Mã chỉ nằm trong bộ nhớ và đổi sau mỗi lần khởi động lại; bearer token không bao giờ mở được trang này. Khi đã có tài khoản, `/ui/setup` trả 404.
- **Lệnh.** `docker compose exec hub kb user add <name> --role admin` (nó hỏi mật khẩu hai lần).
- **Biến bootstrap.** Trong `.env`, đặt `STRATA_KB_BOOTSTRAP_ADMIN=<name>` và `STRATA_KB_BOOTSTRAP_PASSWORD_HASH='<hash>'`, với `<hash>` là kết quả của `kb user hash` (nó hỏi mật khẩu; chạy trên bất kỳ máy nào có strata-kb). **Giữ nguyên dấu nháy đơn**: Docker Compose mở rộng `$` trong `.env`, mà hash chứa nhiều ký tự `$`. Mật khẩu thuần văn bản không bao giờ được chấp nhận. Khi đã có bất kỳ tài khoản nào, cả hai biến bị bỏ qua và không in mã thiết lập.

Vai trò là `admin`, `ba` và `dev`; chỉ `admin` thấy và dùng trang Registry. Thêm tài khoản: mục 3.2.

2.5 Đăng ký các publisher

Intake chỉ nhận publish từ một repository có trong `federation/registry.yaml`, và chính mục đó (không phải payload) quyết định `repo_id` nào repo được ghi. Thêm từng publisher bằng một PR bình thường:

```
repos:
 acme/handbook: handbook                              # plain form: just the repo-id
 acme/safety-docs: {repo_id: safety, kind: kb}        # a child: Document menu
 acme/payments: {repo_id: payments, kind: dev}        # a dev repo: -code and -svc, Code menu
 acme/requirements:                                   # a ba repo: -plan, Ticket menu
   repo_id: acme-requirements
   kind: ba
   auto_merge: true                                   # optional: section 4.3
```

- **Hub trở thành có quản trị (governed).** Khi file không còn rỗng, `kb publish` qua git từ chối mọi repo-id không có ở đây và từ chối push trực tiếp. Mọi repo publish qua git, không chỉ những repo dùng intake OIDC, đều phải được đăng ký. Việc tự publish của chính hub (3.5) được miễn.
- **`repo-id` phải khớp chính xác `repo_id:`** trong `.kb/config.yaml` của repo đó. Qua git, sai lệch bị từ chối trước khi ghi bất cứ thứ gì, và nêu cả hai id. Qua intake, publish vẫn thành công và PR vẫn mở, nhưng `kb publish` của publisher hết giờ chờ một PR mà nó không tìm thấy, và các lần upload mất tính tăng dần (mỗi lần publish thành một lần upload đầy đủ). Repo `ba` có `repo_id: acme-requirements` publish `acme-requirements-plan`.
- **`kind`** là `kb`, `ba` hoặc `dev` và quyết định menu của hub (Document, Ticket, Code). Registry thắng: một publish mà kind của repo ứng với thứ khác bị từ chối (`kb publish` thoát với `kind mismatch`; intake trả `403` với cùng nội dung). Một mục không có `kind` vẫn chạy và kind của chính repo được ghi lại. Trên hub có sẵn, `kb init --kind hub` đề xuất một kind cho mỗi mục chưa có kind, hiện diff rồi hỏi (`--yes` ghi không hỏi). Nó không bao giờ commit: bạn tự mở PR, và chỉ sau khi các publisher tự đọc registry (`kb publish` trực tiếp, hub con) đã chạy version của hub.
- **Để trống `workflow:`.** Intake so nó với `job_workflow_ref` đầy đủ, vốn kết thúc bằng `@refs/tags/kb-publish/<timestamp>`, nên một pin không bao giờ khớp và mọi publish bị từ chối. Một mục chỉ nhận `repo_id`, `kind`, `workflow` và `auto_merge`; khoá nào khác làm registry không hợp lệ.
- **Branch protection trên hub cộng intake OIDC mới là ranh giới bảo mật thật sự.** Registry không xác thực được publisher qua git (remote URL do chính họ khai), nên hãy bảo vệ nhánh mặc định: đó là thứ làm cho đường review có tính ràng buộc.

Bật intake. Tạo một GitHub App (chủ hub) với `Contents: Read and write` và `Pull requests: Read and write`, cài nó chỉ trên repo hub, và tải khoá riêng về. Rồi đặt trên server:

Biến	Giá trị
`STRATA_KB_GH_APP_ID`	id của App
`STRATA_KB_GH_APP_KEY`	**đường dẫn** tới file PEM, server đọc được
`STRATA_KB_INTAKE_AUDIENCE`	URL mà các repo của bạn đặt trong `intake:`, không có dấu `/` ở cuối

Đủ cả ba thì bật `/intake/*`. Thiếu một biến, hoặc PEM không đọc được, thì intake tắt (log ghi `intake disabled`), còn MCP chỉ đọc, API và UI vẫn chạy. Trong Docker, file PEM phải tồn tại bên trong container, mà compose dựng sẵn chỉ mount `./:/data`: hãy mount khoá ở chế độ chỉ đọc trong `docker-compose.override.yml` (`kb init` làm mới `docker-compose.yml`), và đừng bao giờ để nó trong một đường dẫn được theo dõi. Cũng App này push các chỉnh sửa Registry (3.3) và merge PR khi auto-merge bật (4.3).

2.6 Gửi cấu hình cho chủ repo

Mỗi chủ repo cần những thứ sau từ bạn, cùng một mục registry từ 2.5:

Họ cần	Đặt ở đâu	Ghi chú
Git URL hoặc đường dẫn của hub	`hub:` trong `.kb/config.yaml`	**Không bao giờ đặt token trong `hub:` hay `intake:`**: file được commit, nên token đi vào lịch sử (`kb doctor` cảnh báo; viết lại URL và xoay token). URL có credential đi vào biến môi trường `STRATA_KB_HUB`, và máy nào dùng nó cần **Git 2.31 trở lên** (runner CI thì ổn)
URL của intake	`intake:` trong `.kb/config.yaml`	cùng một server; bằng `STRATA_KB_INTAKE_AUDIENCE`
HTTP base URL của hub và bearer token	`kb mcp-setup` (ghi `.env`)	nên dùng prompt ẩn hoặc `STRATA_KB_HTTP_TOKEN` thay cho `--token`, vì cờ đó để token lại trong lịch sử shell và trong danh sách process
Auto-merge có bật cho repo hay không	không ở đâu cả: hãy báo họ	khi tắt, một người trên hub review mọi PR publish

`hub:` (git URL hoặc đường dẫn) và `STRATA_KB_HUB_URL` (HTTP base URL của hub, dùng cho MCP) là hai giá trị khác nhau.

Một version cho mọi repo. Mọi repo trong federation chạy cùng version strata-kb của hub. Nâng cấp hub trước (3.8), rồi báo từng chủ repo chạy `kb doctor` trong repo child, BA hoặc Dev của họ. Khi MCP của họ đã thiết lập (`kb mcp-setup`), `kb doctor` hỏi hub version của nó, cài version đó, đặt các pin `strata-kb==<phiên bản của hub>` trong `.github/workflows/`, và làm mới skill cùng command của họ. Trong CI nó không cài gì và chỉ báo sự lệch. Cái pin quan trọng: một `pip install strata-kb` không ghim trong job có `id-token: write` sẽ để mỗi lần chạy lấy bất cứ thứ gì PyPI đang phục vụ, trong một job có quyền ghi vào hub.

Để xoay token, đặt token mới vào `.env`, tạo lại các dịch vụ (2.3), và báo các chủ repo chạy `STRATA_KB_HTTP_TOKEN=<new> kb mcp-setup`.

2.7 Tùy chọn: lưu asset trên S3

Asset ảnh nằm trong Git (`mode: none`, mặc định) hoặc trong một bucket tương thích S3. Chỉ hub cấu hình việc này. `kb doctor` cảnh báo khi asset trong Git vượt khoảng 100 MB.

1. Tạo một bucket **private** (chặn truy cập công khai) và credential chỉ có quyền Get, Put và Head trên prefix `assets/`. Xuất chúng qua chuỗi biến môi trường AWS chuẩn (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, region); chúng không bao giờ nằm trong `config.yaml`. Với MinIO, R2 hay host tương thích S3 khác, hãy đặt `endpoint` thay vào đó.
2. `pip install "strata-kb[s3]"` trên mọi máy publish hoặc phục vụ.
3. `kb init --kind hub --assets s3`, hoặc sửa khối này bằng tay:

```
# .kb/config.yaml
asset_store: {mode: s3, bucket: my-kb-assets, region: eu-west-1, prefix: assets/}
```

4. `kb assets migrate` chuyển các asset đang có trong Git vào bucket, lần lượt từng repo-id, và commit cục bộ; nó không bao giờ push, nên hãy push như mọi commit khác của hub. `kb assets verify` kiểm tra mọi bản ghi đều phân giải được, mọi tham chiếu markdown đều được phủ và không có gì mồ côi. `kb doctor` xác nhận kho truy cập được.

Từ đó, mỗi lần publish upload ảnh lên bucket thay vì commit chúng, và `/assets/<name>` phục vụ chúng với một cache đĩa cục bộ. Để quay lại, tải từng asset được nêu trong các file `_assets.yaml`, xoá các file đó, đặt `mode: none` rồi commit.

2.8 Tùy chọn: nối các hub (đa tầng)

Một repo `kind: hub` có thể tự khai báo `hub:` (hub cấp trên) và `repo_id:`. Publish nó sẽ mirror **`federation/`** của nó (không phải `.kb/` của chính nó) vào `federation/<hub-id>/` ở cấp trên, giữ nguyên bố cục lồng nhau; id trở thành đường dẫn (`mid/repo-x:doc-id`), độ sâu không giới hạn. Hãy chọn hub mà mọi người truy vấn: hub của một team trả tri thức của team, hub gốc trả tất cả.

- **Chỉ `kb` đi lên.** Hub con có đủ ba menu và nhận publish `ba` và `dev`, nhưng chỉ gửi lên cấp trên các leaf `kb` của chính nó. `kb publish` in một dòng `skipped <leaf> (kind <kind> stays on this hub)` cho mỗi leaf bị giữ lại. Hub cha không bao giờ phục vụ leaf không phải `kb` nằm dưới một hub con, và `kb doctor` nêu tên nó.
- Publish hub-sang-hub cần truy cập Git trực tiếp lên cấp trên, hoặc `gh` cho `--pr`; đường intake chỉ dành cho repo child, BA và Dev. Một chuỗi sẽ quay vòng bị từ chối: `federation cycle detected`. Một `federation/` cục bộ bị làm rỗng không lan truyền việc xoá lên cấp trên (publish từ chối thay vì xoá sạch các mục); xoá một số mục thì có lan truyền, như một mirror thông thường trong PR trên hub cấp trên.

---

3. Công việc hằng ngày

Việc	Lệnh	Kết quả
Review một PR publish	trên Git host của bạn; xem 3.1	merge nó là cái cổng
Thêm, đổi hay xoá tài khoản	`kb user` (`add`, `passwd`, `role`, `remove`, `list`)	`.kb-auth/users.db`
Đăng ký repo, bật tắt auto-merge	trang Registry (`/ui/registry`), hoặc PR trên `federation/registry.yaml`	repo được phép publish
Nhả claim bị kẹt, đọc Q&A	trang ticket, `/ui/questions`	3.4
Biên soạn trên hub	các bước trong tài liệu Child, rồi `kb publish`	tài liệu trong `federation/<repo-id>/`
Tìm kiếm	`kb query`, `kb get`, `kb tags`, Web UI	các section có trích dẫn
Kiểm tra sức khoẻ	`kb doctor`	các dòng `[error]`, `[warning]`, `[info]`
Sửa index	`kb reindex`	`federation/index.yaml` được dựng lại
Nâng cấp	3.8	mọi repo cùng một version

3.1 Review và merge PR publish

Mỗi lần publish đến như một PR trên hub này, sửa `federation/<repo-id>/` và `federation/index.yaml` tổng hợp. Merge nó là cái cổng: sau khi merge, hub pull commit mới (4.1) và nội dung tìm kiếm được. Mỗi loại repo gửi gì, và điều gì đã được kiểm tra trước khi nó tới tay bạn:

Từ	Bạn nhận	Đã kiểm tra
Child (và hub của bạn)	tài liệu: đủ bốn lớp (L0 đến L3)	không còn phát hiện L3 nào chưa xác nhận (không thì publish từ chối); section chưa duyệt chỉ cảnh báo, trừ khi publisher dùng `--require-reviewed`
Repo BA	`<repo-id>-plan`: các mission và các ticket **ready** của nó	Definition of Ready (`kb plan-ingest` bỏ qua phần còn lại)
Repo Dev	`<repo-id>-code` (do `kb code-ingest` sinh) và `<repo-id>-svc` (do người biên soạn)	`-svc` được review và duyệt trong repo của chính Dev trước khi publish, nên một bản sửa `-svc` đi kèm không được review lại ở đây

Với một PR tài liệu, hãy đọc diff như một tài liệu có theo dõi thay đổi. Kiểm tra văn xuôi mới (các file `.md`, không phải `.raw.md`) so với hiểu biết của bạn về chủ đề: không thiếu gì quan trọng, không có gì mà nguồn không nói. Kiểm tra mọi mã, định danh, con số và đơn vị đúng y như trong nguồn. Đừng kiểm tra bảng từng dòng, vì `kb build` đã so sánh chúng; chỉ liếc để tìm lỗi trích xuất (một hàng hay cột mà bộ đọc PDF đọc sai), thứ máy không bắt được. Kiểm tra các tóm tắt một dòng trong `_manifest.yaml` nói đúng mỗi section nói về gì, đủ để tìm kiếm thấy nó. Nếu có gì sai, sửa `.md` hoặc `.yaml` trong trình soạn thảo web của Git host, ghi chú lý do, hoặc nhờ publisher sửa.

Một PR đã đóng hoặc đã merge mà nhánh `publish/<repo-id>` của nó đã bị xoá thì không sao: lần publish tiếp theo mở một PR mới.

3.2 Quản lý tài khoản

```
docker compose exec hub kb user add <name> --role ba   # prompts for the password twice
docker compose exec hub kb user passwd <name>          # also: role <name> <role>, remove <name>
docker compose exec hub kb user list                   # never prints hashes
```

Vai trò là `admin`, `ba` và `dev`. Admin cuối cùng không thể bị xoá hay hạ quyền. Xoá `.kb-auth/session.key` sẽ đăng xuất mọi người. Ngoài Docker, chạy các lệnh tương tự với `--kb <hub>/.kb`. Bearer token không phải là tài khoản: trong UI nó chỉ đọc và không bao giờ mở được trang Registry.

3.3 Registry và auto-merge

Trang Registry (`/ui/registry`, chỉ `admin`) thêm hoặc xoá một repo và bật tắt **auto-merge** của nó. Mỗi chỉnh sửa là một commit trên nhánh mặc định của hub, được push bằng GitHub App của intake (thông điệp `registry: <action> <repo> (by <user>)`) và không bao giờ force-push. Nếu nhánh mặc định được bảo vệ, App phải được phép push vào nó, nếu không mọi chỉnh sửa bị từ chối. Trang chỉ đọc, kèm một banner nêu lý do, khi intake chưa cấu hình hoặc hub không có Git remote. Xoá một repo không xoá nội dung đã publish của nó. Sửa `federation/registry.yaml` bằng PR (2.5) cũng làm được việc đó và không cần App.

3.4 Câu hỏi và claim

Q&A của ticket. Một Dev gặp câu hỏi nghiệp vụ về ticket đã publish sẽ hỏi BA qua các skill; BA trả lời; hub giữ cuộc trao đổi. Bạn không cấu hình gì: kho được tạo ở lần gọi Q&A đầu tiên (6.5). Menu **Q&A** chỉ đọc (`/ui/questions`) liệt kê các ticket có câu hỏi, nhóm theo repo BA, với số lượng open, waiting và answered (`?all=1` gồm cả ticket đã trả lời hết), và `/ui/questions/ticket?repo=<repo-id>&ticket=<id>` cho thấy cuộc trao đổi của một ticket.

Claim. `kb claim take` của một Dev giữ hai Dev khỏi cùng một ticket. Một claim hết hạn sau `STRATA_KB_CLAIM_TTL` giây không được cập nhật (mặc định `14400`, 4 giờ); claim ở `review` hoặc `publishing` có PR, và claim `stopped`, không bao giờ hết hạn. Admin của hub nhả claim của bất kỳ ai bằng nút **Release**, kèm lý do, trên trang ticket của menu Ticket.

3.5 Biên soạn ngay trên hub

Hub ingest, tóm tắt, build và duyệt tài liệu bằng đúng các lệnh như child: hãy theo tài liệu Child. Điểm khác:

- `hub:` là `.`, nên `kb publish` là một **self-publish**: nó mirror `.kb/` vào `federation/<repo-id>/` ngay trên repo này. Cho tới lúc đó server không tìm được tài liệu. Self-publish được miễn registry. Có Git remote thì nó mở PR (cần `gh` dùng được); không có remote thì nó commit thẳng.
- `kb ingest` cũng chạy được trong container: `docker compose run --rm hub kb ingest …`.
- Tài liệu mà Markdown nay là bản gốc được gắn tag bằng `kb authored <doc>` (`--off` gỡ tag); `kb ingest` sau đó từ chối ingest lại nó, để một PDF không ghi đè các chỉnh sửa tay. `kb summarize --redo` vẫn được phép.
- PDF nguồn ở trong `source/`, thư mục Git bỏ qua. Đừng commit chúng.

3.6 Tìm kiếm và Web UI

```
kb query "parental leave eligibility"
kb query "parental leave" --tags hr --budget 600 --kind kb
kb get hr-handbook 4.12 --level l2     # or --level l3
kb tags                                # every tag published on the hub
kb stats                               # token size per layer, per document
```

Trên hub, các lệnh này chỉ đọc `federation/`. Mọi kết quả trích dẫn `<repo-id>:<doc-id> §<section> (<revision>)`. Hãy truy vấn bằng ngôn ngữ của chính tài liệu: kho không bao giờ dịch. `--semantic` ép dùng nhánh semantic và cảnh báo khi thiếu extra `embed`.

Thanh trái của Web UI là **Overview, Document, Ticket, Code, Q&A**: Document hiện các leaf `kb`, Ticket hiện các plan BA (một trạng thái cho mỗi story), Code hiện `-code` và `-svc` tại `/ui/code`. Ô tìm kiếm trên cùng tìm mọi loại trong một danh sách.

3.7 Kiểm tra sức khoẻ

```
kb doctor      # exit 1 on an error, 2 when the hub cache is stale
kb reindex     # rebuild federation/index.yaml when it has drifted; --force also rebuilds search
```

`kb doctor` báo nội dung hỏng và file thiếu, nội dung đã publish bị sửa tại chỗ trên hub, những file mà một lần publish sẽ không bao giờ ghi, một bản clone hub lưu credential trong `.git/config` của nó, và Git cũ hơn 2.31. Trên hub nó còn kiểm tra kind: mục registry không có `kind` (cảnh báo; sửa bằng `kb init --kind hub`), một leaf có kind đã ghi mâu thuẫn với registry, và leaf không phải `kb` nằm dưới một hub con. `federation/index.yaml` ghi `content_sha256` cho mỗi snapshot nên việc can thiệp trái phép phát hiện được. Hãy chạy `kb doctor` sau mỗi lần nâng cấp: các kiểm tra chặt hơn, và mỗi thông báo nêu lệnh sửa nó.

3.8 Nâng cấp hub

Nâng cấp hub trước; các repo theo sau (2.6).

```
pip install --upgrade strata-kb
kb init --kind hub        # refreshes the compose image tag, skills and workflows
docker compose pull && docker compose up -d
kb doctor
```

Commit những gì `kb init` đã đổi. Rồi báo chủ của mọi repo child, BA và Dev chạy `kb doctor`. `kb doctor` không đồng bộ chính repo hub: `kb init` mới là cách nó được nâng.

---

4. Các chế độ tự động

Hub không có `kb auto`. Những gì chạy mà không cần bạn:

Chế độ	Làm gì	Công tắc
Tự động pull	dịch vụ `hub` pull Git checkout để PR đã merge lên sóng	`STRATA_KB_HUB_PULL_INTERVAL`
Publish CI qua OIDC	CI của chính một repo publish lên hub này mà không lưu secret	ba biến intake (2.5)
Auto-merge	intake tự merge PR publish của một repo	theo từng repo: trang Registry, hoặc `auto_merge: true`
Code-sync	dịch vụ `hub-sync` đánh dấu tài liệu hub nêu tên code đã đổi	`kb code-sync`, `/kb-code-sync`

4.1 Tự động pull

Server HTTP chạy `git pull --ff-only` trên hub checkout cứ mỗi `STRATA_KB_HUB_PULL_INTERVAL` giây (mặc định `300`; `0` là tắt) và đồng bộ lại search index khi HEAD đã di chuyển, nên một PR đã merge tìm kiếm được mà không cần khởi động lại. Một checkout bị sửa tay hoặc phân kỳ không bao giờ bị động tới: lần pull thất bại, một cảnh báo được ghi log, và server tiếp tục phục vụ những gì nó có. Hãy sửa trong một bản clone khác.

4.2 Publish CI qua OIDC

`kb ci-publish` là đường CI: nó chạy trong job GitHub Actions của chính publisher, xác thực với intake của hub này bằng GitHub Actions OIDC, và không cần secret.

1. Một người chạy `kb publish` trong repo, lệnh gắn tag `kb-publish/<timestamp>` cho nội dung đã commit và push tag.
2. Tag khởi động workflow, workflow chạy `kb ci-publish` với `id-token: write`.
3. Intake xác minh token, kiểm tra registry, và mở PR trên hub này (và merge nó, nếu auto-merge bật cho repo). Lệnh của publisher thăm dò liên kết PR và hết giờ sau 600 giây.

Intake chỉ nhận `refs/tags/kb-publish/*`. Sự kiện nào tạo ra tag như vậy:



Với auto-merge của Registry bật cho một repo BA, một merge của BA vì vậy tới hub này mà không có review phía hub: hãy áp quy tắc review của bạn lên `main` của repo đó. Để trống `workflow:` cho tất cả (2.5).

4.3 Auto-merge theo từng repo

Ngay sau khi intake mở PR publish của một repo, nó merge PR đó (một merge commit, hoặc squash khi repo cấm merge commit). Nó merge **toàn bộ** PR, kể cả nội dung `-svc` đi kèm. App cần quyền ghi `Contents` và `Pull requests` (2.5). Nếu merge thất bại, PR vẫn mở cho người xử lý và publish vẫn tính là xong (`merged: false` cùng lý do trong `detail`). `kb publish` và `kb ci-publish` chỉ in "merged" khi nó đã merge; bản tóm tắt bước CI vẫn hiện `hub PR: <url>`. Chỉ bật cho những repo mà bạn không cần review nội dung ở đây.

4.4 Code-sync

Một tài liệu hub viết tay hoặc ingest từ PDF (kiến trúc, tiêu chuẩn, thiết kế) bị lỗi thời khi các tài liệu `-code` hay `-svc` mà nó nêu tên thay đổi. Code-sync tìm các tài liệu đó và giữ chúng đúng, không cần model trong job hằng ngày.

1. **Đánh dấu các tài liệu.** `kb authored <doc>` cho từng tài liệu mà Markdown nay là bản gốc. Tài liệu gắn tag `code-sync-skip` bị bỏ khỏi lần quét. `kb code-sync scan` in một dòng `note:` cho mỗi tài liệu hub chưa `authored`.
2. **Đặt baseline một lần.** `kb code-sync baseline` ghi commit hub mà code-sync đang cập nhật tới, trong `kb-code-sync.yaml` ở gốc hub. Commit nó vào nhánh mặc định. `--at <sha>` ghi một commit khác.
3. **Phát hiện hằng ngày.** Dịch vụ `hub-sync` chạy `kb code-sync run --issue --loop`: quét, cập nhật issue, ngủ `STRATA_KB_CODE_SYNC_INTERVAL` giây (mặc định `86400`, tối thiểu `60`). Nó không bao giờ pull, commit hay push (dịch vụ `hub` pull `/data`). Nó so các tài liệu `-code` và `-svc` giữa baseline và đầu nhánh mặc định và liệt kê các section của hub nêu tên thứ được thêm, xoá hay đổi. Một issue gắn nhãn `kb-code-sync` luôn mở khi còn phát hiện: chỉ được viết lại khi chúng đổi, đóng kèm một bình luận khi lần quét không thấy gì. Một lỗi quét không bao giờ đóng nó.
4. **Nhịp sprint.** Quản lý hub chạy `/kb-code-sync` trong agent của họ bất kỳ lúc nào issue đang mở. Skill chạy lại lần quét, đánh giá từng phát hiện (sửa, không cần đổi, hoặc cần SA), sửa các section, tóm tắt lại, chạy `kb build --strict`, dời baseline, và publish bằng `kb publish --branch kb-bot/code-sync`: một PR cuốn chiếu mà các lần chạy sau bổ sung thêm. SA hoặc quản lý hub review nó ở buổi review sprint, có thể chạy `kb approve`, và merge. Skill không bao giờ merge và không bao giờ duyệt.

Các phát hiện thuộc `removed` (code đã mất, văn xuôi nhiều khả năng sai), `changed`, `added` và `image`, theo thứ tự đó. `kb code-sync issue` đánh giá các issue đang mở (`current`, `stale`, `duplicate`, `foreign`); skill chỉ làm việc từ issue dùng được và không bao giờ sửa, đóng hay bình luận trên một issue.

Điều kiện cần	Vì sao
`origin/HEAD` đã đặt (`git remote set-head origin --auto`)	lần quét đọc đầu nhánh mặc định; hub có remote mà không có `origin/HEAD` thoát với `1`
GitHub App có `issues: write`, `pull_requests: read`, `contents: read`	`hub-sync` gọi REST API của issue và đọc baseline từ repo hub. App của intake cần được thêm các quyền này và admin của tổ chức duyệt lại bản cài đặt
hoặc `STRATA_KB_GH_TOKEN`	một token có cùng ba quyền, chỉ dùng khi App chưa cấu hình
`gh`, đã đăng nhập, trên máy của quản lý	`kb publish --branch` cùng tiêu đề và nội dung PR

Không có credential, `hub-sync` vẫn quét và ghi log work order, ghi một dòng rằng issue chưa được cập nhật và vì sao, rồi tiếp tục chạy.

---

5. Xử lý sự cố

Triệu chứng	Cách xử lý
Server thoát: `STRATA_KB_HTTP_TOKEN is the placeholder`, `… characters; at least 32 are required`, hoặc `must be printable ASCII`	`kb docker-setup --force`, hoặc đặt token từ 32 ký tự ASCII in được trở lên, không có dấu cách (2.2)
Server thoát vì `STRATA_KB_ALLOWED_HOSTS entry … is not a host name`, hoặc vì `STRATA_KB_TRUSTED_PROXIES`	Chỉ tên host trần, phân cách bằng dấu phẩy; một số nguyên từ 0 trở lên (2.3)
`/mcp` trả `421 Invalid Host header`, `/api` và `/ui` vẫn chạy	Đặt `STRATA_KB_ALLOWED_HOSTS` bằng tên host công khai (2.3)
Một lần đăng nhập sai khoá mọi người, hoặc một người gọi làm cạn giới hạn tốc độ	Proxy che mất người gọi thật: đặt `STRATA_KB_TRUSTED_PROXIES` (2.3)
Mọi trang `/ui` chuyển hướng sang `/ui/setup`	Chưa có tài khoản nào (2.4). Sau tài khoản đầu tiên, `/ui/setup` là 404
Trang Registry chỉ đọc, kèm banner	Intake chưa cấu hình (2.5), hoặc hub checkout không có Git remote `origin`
Chỉnh sửa Registry bị từ chối	Nhánh mặc định được bảo vệ và App không được phép push: hãy cho phép
Log ghi `intake disabled`; không có `/intake/*`	Thiếu một trong ba biến, hoặc PEM ở `STRATA_KB_GH_APP_KEY` không đọc được. Trong Docker, đường dẫn phải tồn tại bên trong container
Publisher: `is not registered`, HTTP 403	Thêm `owner/repo` dưới `repos:` trong `federation/registry.yaml` (2.5)
Publisher: `pinned to workflow … in federation/registry.yaml`	Bỏ pin `workflow:`
`kind mismatch: registry says …`	Sửa `kind:` trong mục registry hoặc trong `.kb/config.yaml` của repo; kind của registry thắng
`kb publish` của một publisher hết giờ nhưng PR đã có	`repo-id` trong registry khác `repo_id:` của repo; sửa một trong hai (2.5)
Publisher qua git: bị từ chối push trực tiếp, hoặc repo-id không có trong danh sách	Hub có quản trị: đăng ký repo, và publish bằng PR
Upload bị từ chối với `400`, đường dẫn quá dài	Đường dẫn bị giới hạn 110 đơn vị mã UTF-16 để `federation/<repo-id>/<path>` vừa với client Windows; rút ngắn tên tài liệu hoặc tên file
Đã merge trên repo của publisher, chưa tìm kiếm được	Nó tìm được sau khi PR **hub** được merge và hub pull (tối đa `STRATA_KB_HUB_PULL_INTERVAL` giây). Xem log server có lần pull thất bại không: checkout bị sửa tay hoặc phân kỳ bị bỏ qua
Nội dung đến mà không có review của hub	Auto-merge đang bật cho repo đó (4.3), có thể sau một merge của BA vào `main` (theo bản dựng khung, 4.2)
`kb doctor` nêu tên file nằm trong một mục federation	Một lần publish cũ đã mirror nhiều hơn các artefact của `.kb/`. Publish lại để loại chúng, và **xoay mọi credential chúng chứa**: xoá file không xoá nó khỏi lịch sử Git
`kb summarize`, `kb build` hoặc `kb publish` từ chối một tài liệu hub: `L3 review pending`	Một phát hiện L3 chưa được xác nhận: `kb l3 report <doc-id>`, kiểm tra từng section với PDF, rồi `kb l3 confirm <doc-id> --section <id>` (hoặc `--all`); xem tài liệu Child
Mọi lệnh hub lỗi xác thực	`git --version`: dưới 2.31 với URL hub có credential thì không gì chạy. Nâng cấp Git
`Q&A needs the shared hub server — point MCP at it (kb mcp-setup)`	MCP của client là stdio server của repo Dev hoặc một bản clone cache của URL hub; trỏ nó vào `/mcp` của server này (6.5)
`hub-sync` ghi log rằng issue chưa được cập nhật	Không có credential App hay `STRATA_KB_GH_TOKEN`, hoặc thiếu quyền (4.4)

6. Phụ lục

6.1 Biến môi trường

Biến	Ý nghĩa
`STRATA_KB_HTTP_TOKEN`	bearer token của server; bắt buộc cho `--transport http`; từ 32 ký tự ASCII in được trở lên, không phải `change-me`
`STRATA_KB_HUB`	ghi đè `hub:`; `--hub` thắng nó. Mang URL có credential, thứ không bao giờ đặt trong `hub:`
`STRATA_KB_ALLOWED_HOSTS`	tên host trần mà `/mcp` chấp nhận ngoài localhost, phân cách bằng dấu phẩy
`STRATA_KB_TRUSTED_PROXIES`	số reverse proxy tin cậy, số nguyên từ 0 trở lên, mặc định `0`
`STRATA_KB_HUB_PULL_INTERVAL`	số giây giữa các lần pull tự động, mặc định `300`, `0` = tắt
`STRATA_KB_HUB_CACHE`, `STRATA_KB_HUB_TTL`	cache clone mà một lệnh dùng để đọc URL hub (mặc định `~/.strata-kb/hub/`) và thời gian nó còn được coi là mới (mặc định 900 giây)
`STRATA_KB_GH_APP_ID`, `STRATA_KB_GH_APP_KEY`, `STRATA_KB_INTAKE_AUDIENCE`	intake: id App, đường dẫn tới PEM, audience OIDC mong đợi; đủ cả ba hoặc không biến nào
`STRATA_KB_GH_TOKEN`	token cho issue của code-sync khi chưa cấu hình App
`STRATA_KB_CODE_SYNC_INTERVAL`	số giây giữa các lần quét của `hub-sync`, mặc định `86400`, tối thiểu `60`
`STRATA_KB_BOOTSTRAP_ADMIN`, `STRATA_KB_BOOTSTRAP_PASSWORD_HASH`	admin đầu tiên khi chưa có tài khoản nào; hash là kết quả của `kb user hash`; mật khẩu thuần văn bản không bao giờ được chấp nhận
`STRATA_KB_CLAIM_TTL`	số giây một claim ticket sống mà không được cập nhật, mặc định `14400`; giá trị không dương sẽ cảnh báo và quay về mặc định
`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, …	credential S3, từ chuỗi AWS chuẩn; không bao giờ trong `config.yaml`

6.2 Khoá cấu hình, registry và nơi lưu

Hub đọc các khoá này từ `.kb/config.yaml`: `kind: hub`, `hub` (`.` cho hub gốc; một đường dẫn hay URL khác nối nó lên cấp trên, 2.8), `repo_id`, `asset_store` (`mode`, `bucket`, `region`, `endpoint`, `prefix`; prefix mặc định là `assets/`), `l3_fix.enabled` (tài liệu Child), và `intake` (bị từ chối với publish hub-sang-hub, được phép với self-publish). `llm:` trong `.kb/index.yaml` gồm `runner`, `model`, `effort`, `max_workers` và `timeout`. Ở gốc hub, `kb-code-sync.yaml` giữ baseline của code-sync. Một mục registry là `owner/repo: repo-id` hoặc `{repo_id, kind, workflow?, auto_merge?}`; `auto_merge` mặc định là `false`.

Đường dẫn	Chứa	Được commit
`federation/`	các snapshot đã publish, `index.yaml`, `registry.yaml`	có
`.kb-auth/` (`users.db`, `session.key`)	tài khoản và khoá cookie; xoá `session.key` sẽ đăng xuất mọi người	**không bao giờ**
`.kb-qa/qa.db`	Q&A của ticket	**không bao giờ**
`.env`, `.kb-work/`, `source/`	token; search index và dữ liệu tạm; PDF nguồn	không

6.3 Các công cụ MCP

Server cung cấp mười bốn công cụ. Chỉ `kb_question_ask` và `kb_question_answer` ghi.

Công cụ	Mục đích
`kb_search`	tìm section bằng ngôn ngữ tự nhiên; trả L2 trong một ngân sách token, kèm trích dẫn (`query`, `tags`, `budget`, `kinds`). Mỗi nhánh bị giới hạn 50 kết quả trước khi hợp nhất, và công cụ báo khi có kết quả bị bỏ. Nó cũng báo khi hai kết quả đầu là cùng một section do hai repo publish, khi điểm từ khoá của chúng bằng nhau tuyệt đối, hoặc khi cả hai đều được hybrid xác nhận và nằm trong ngưỡng nhập nhằng
`kb_get_section`	lấy đúng một section ở L2 hoặc L3
`kb_context_new`	ghim một khối trích dẫn `kb-context` tại commit hiện tại của hub, từ các ref đã xác nhận
`kb_resolve`	phân giải một khối `kb-context` (hoặc một ticket chứa nó): các section ở version đã ghim cùng độ mới của chúng
`kb_ticket_lint`	cổng Definition-of-Ready cho một ticket nháp
`kb_ticket_check`	cổng SA grounding đối chiếu tài liệu `-code` của hub
`kb_ticket_stale`	báo cáo trích dẫn cũ cho một ticket hay mission: trích dẫn mà hub đã sửa hoặc làm mất kể từ khi được ghim, và các section `-code` đã dời
`kb_mission_lint`	cổng Definition-of-Ready cho mission
`kb_ticket_fetch`	một ticket BA đã publish, đọc từ `<ba-repo>-plan` của hub theo id
`kb_question_ask`	**ghi.** Hỏi BA một câu về ticket đã publish; trả `Q<id>`
`kb_question_list`	liệt kê câu hỏi của ticket với trạng thái suy ra (`open`, `waiting`, `answered`, hoặc `all`)
`kb_question_answer`	**ghi.** Ghi lại câu trả lời của BA cho một câu hỏi đang mở
`kb_mission_next`	story nào tiếp theo, từ các plan đã publish trên hub
`kb_doctor_hub`	phần phía hub của `kb doctor` cho repo child, BA hoặc Dev (version, trạng thái publish, kind đã ghim); do CLI gọi, không dành cho agent

Skill BA gọi mọi công cụ trừ `kb_ticket_fetch`, `kb_question_ask` và `kb_doctor_hub`; skill Dev gọi `kb_search`, `kb_get_section`, `kb_context_new`, `kb_resolve`, `kb_ticket_lint`, `kb_ticket_check`, `kb_ticket_fetch`, `kb_question_ask` và `kb_question_list`.

6.4 Những gì server thực thi

- Mọi phản hồi mang CSP (`script-src 'self'`), `X-Frame-Options: DENY`, nosniff, `Referrer-Policy` và `Permissions-Policy`; HSTS chỉ khi yêu cầu tới server qua https.
- Cookie phiên `/ui` là một giá trị có chữ ký, có hạn, được ký bằng `.kb-auth/session.key`, không bao giờ bằng bearer token (thứ mà mọi repo đều giữ). Nó mang tên người dùng, không mang vai trò. Thân yêu cầu đăng nhập bị giới hạn 8 KiB dù được gửi bằng cách nào.
- Ba bộ giới hạn tốc độ theo từng người gọi: `/intake/publish` 30 mỗi phút, `/intake/manifest` và `/intake/status` 120 mỗi phút (dùng chung), và `/ui/login` 5 mỗi phút. Một lần thử `Authorization` thất bại dùng chung xô với đăng nhập, nên không thể né việc khoá bằng cách dùng header. `STRATA_KB_TRUSTED_PROXIES` quản cả ba.
- `GET /intake/status` công khai có chủ đích (CLI không cần secret thăm dò nó) và, khi không có token, chỉ trả `state` và `pr_url`, khoá theo repo-id và commit: cùng giả định mạng nội bộ như MCP. `detail` của lỗi chỉ đến người gọi xuất trình OIDC token của repo đó.
- Branch protection trên hub cộng intake OIDC là ranh giới bảo mật thật sự (2.5): registry không xác thực được publisher qua git.
- Publish chỉ sao chép **artefact của `.kb/`**: tài liệu, index, manifest và `assets/`. File cấu hình và dotfile bị giữ lại kèm một dòng `[warn] … (allowlist)`, nên một `config.yaml` mà ai đó để credential vào không bao giờ tới hub. `kb doctor` cảnh báo khi một `config.yaml` được theo dõi mang credential.
- `docs/deploy-remote-mcp.md` nói về khoảng hở TLS/HSTS và rủi ro khi tin proxy (2.3).

6.5 Quy tắc Q&A

Kho là SQLite tại `<hub root>/.kb-qa/qa.db` (trong Docker là `/data/.kb-qa/qa.db`), được tạo và git-exclude ở lần gọi Q&A đầu tiên. **Không bao giờ commit `.kb-qa/`**: `.gitignore` của hub mới đã liệt kê nó, và hub cũng thêm nó vào `.git/info/exclude` khi web UI khởi động. Sao lưu nó cùng phần còn lại của volume hub.

Ba công cụ chỉ chạy với chính hub: endpoint HTTP của server này, hoặc stdio server chạy bên trong checkout của chính hub (`--kb <hub>/.kb`). Mọi đích khác bị từ chối: stdio server của một repo Dev (kể cả cái trỏ tới đường dẫn của hub) hay một URL hub đọc qua bản clone cache cục bộ trả `Q&A needs the shared hub server — point MCP at it (kb mcp-setup)`, vì một kho ở đó sẽ không phải kho dùng chung. Repo BA và Dev trỏ MCP tới server HTTP bằng `kb mcp-setup`.

Bất kỳ người gọi nào hub xác thực đều có thể hỏi và trả lời: bearer token, hoặc một phiên của vai trò bất kỳ. `asked_by` và `answered_by` là tên do người gọi tự gõ (các skill truyền `git config user.name`): để theo dõi, không phải xác thực. Giới hạn: câu hỏi và câu trả lời 4000 ký tự, context 1000, tên 100 (một dòng in được, không có ký tự điều khiển), tối đa 50 section thay đổi cho mỗi câu trả lời, và tối đa 50 câu hỏi mở cho mỗi ticket. Một câu hỏi là *open* cho đến khi được trả lời; một câu trả lời làm đổi ticket vẫn là *waiting* (hiện là "waiting for publish") cho đến khi revision ticket mới của BA có trên hub, rồi nó là *answered*.

Runner tự động của repo `ba` hoặc `dev` thăm dò `GET /api/questions` bằng cùng bearer token. Nó chỉ đọc và trả `{"questions": [...]}`. Tham số: `repo`, `ticket`, và `status` (mặc định `open`, hoặc `waiting`, `answered`, `all`; giá trị khác là `400`). Hub chưa có kho Q&A trả về danh sách rỗng; hub không phục vụ chính checkout của nó trả `503`.

6.6 Mã thoát

`0` ổn, `1` lỗi hoặc cấu hình sai, `2` một trích dẫn cũ (với `kb doctor` là cache hub cũ) hoặc lỗi cách dùng CLI. `kb code-sync scan` và `run`: `0` không có gì cần làm, `2` có phát hiện, `1` lỗi; `kb code-sync issue`: `0` có issue dùng được, `2` không có, `1` lỗi. Mã `2` cho trích dẫn cũ đến từ `kb resolve`, `kb doctor --context`, `kb ticket lint --fail-on-stale` và `kb mission lint --fail-on-stale`; `kb doctor` cũng thoát `2`, dù có hay không có `--context`, khi chính cache hub đã cũ.